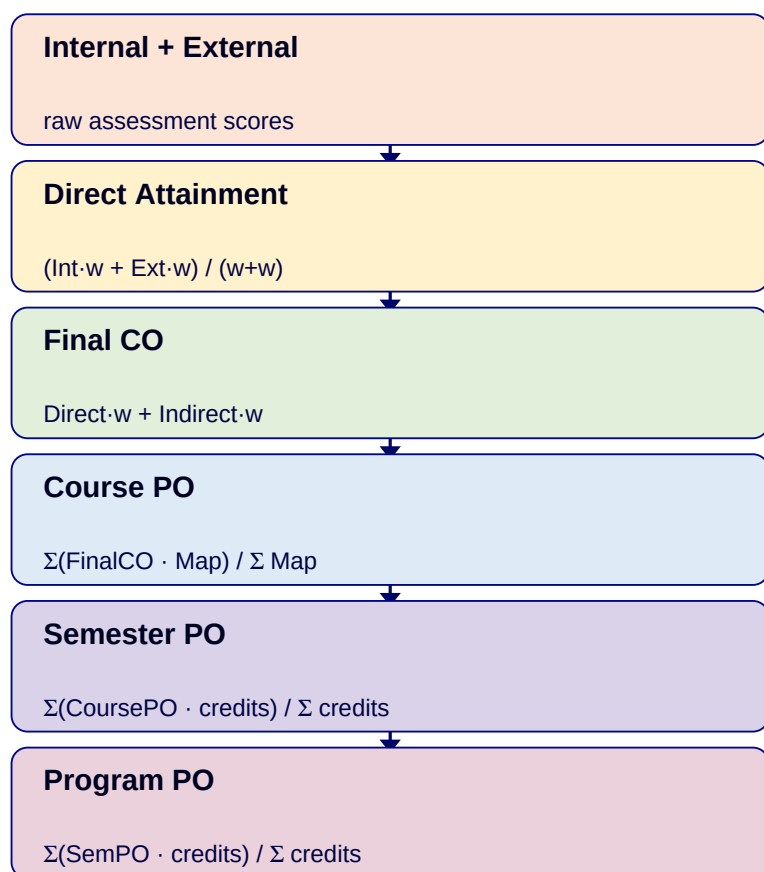


CO-PO Attainment Workbench

Visual User Guide



A step-by-step walkthrough of Stages 1 through 4 — from CO/PO mapping all the way to program-level outcome attainment.

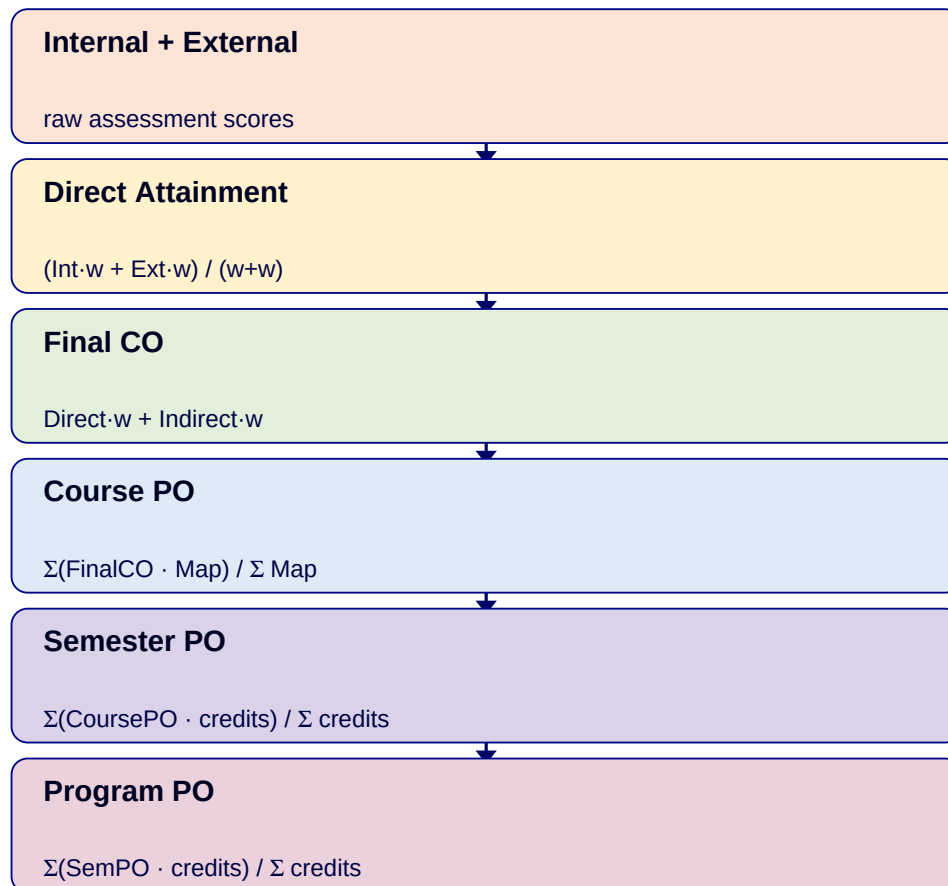
1. What this tool does

The workbench takes course assessment data and turns it into the four reports an OBE-accredited program needs:

Final CO Attainment · Course-wise PO Attainment · Semester-wise PO Attainment · Program-level PO Attainment

Every stage is the same shape: a **weighted average**. Only the weights change as you climb the funnel.

The funnel at a glance



Each box's output is the next box's input. The Streamlit app mirrors this with four steps you walk left → right.

2. Getting started

Option A — Streamlit Cloud

Point Streamlit Cloud at the repo and let it auto-install. The included `requirements.txt` pulls in streamlit, sentence-transformers, transformers, and torch so all backends work out of the box.

Field	Meaning
Repository	ShivamJoshii/phd-copo
Branch	main (post-merge) — or the feature branch on PR #27
Main file	streamlit_app.py
Python	3.10 or 3.11
Secrets (optional)	HF_TOKEN — avoids HF Hub rate limits on cold start

Option B — Local

```
python -m venv .venv
source .venv/bin/activate
pip install -e ".[ui,nlp]"
streamlit run streamlit_app.py
```

The app opens at `http://localhost:8501`. You'll see four tabs across the top — that's the flow.

Sample data lives in examples/

Field	Meaning
co.json / po.json	Step 1 mapping inputs
co_attainment_spec.json	Step 2 CO scores (spec worked example)
attainment_config_spec.json	Step 2 weights (30/50, 80/20)
mapping_matrix_spec.csv	Step 2 CO×PO map (worked example)
courses_semester1.csv	Step 3 — two courses, DBMS + OS
semesters_program.csv	Step 4 — two semesters

3. Step 1 — CO-PO Mapping

What this step produces

A pairwise mapping matrix: for every Course Outcome × Program Outcome pair, a strength score on a 0/1/2/3 scale.

CO-PO Mapping & Attainment Workbench

Step 1 Inputs

[Upload file]
[Upload file]
Backend: tfidf v
Model: (default)

Step 1: Mapping

Step 2: Course

Step 3: Semester

Step 4: Program

Step 1 — CO-PO Mapping

Upload CO + PO files (JSON or CSV) in the sidebar.
Pick semantic backend: tfidf / sbert / bert.

Run Mapping

matrix preview appears here (color-coded 0–3)

UI mockup of Step 1

Inputs

Field	Meaning
CO file	JSON or CSV with columns: CO, description
PO file	JSON or CSV with columns: PO, description (PSOs allowed as PSO1, PSO2...)
Semantic backend	tfidf (default) · sbert · bert

Sample output — the mapping matrix

CO\PO	PO1	PO2	PO3	PO4
CO1	3	2	0	1
CO2	2	3	1	0
CO3	1	2	3	2

Color scale — 0 none · 1 weak · 2 medium · 3 strong.

Downloads

pair_predictions.csv — every CO×PO pair with strength, confidence, explanation, and which semantic method was used.

matrix.csv — the CO×PO grid only (this is what flows to Step 2).

4. Step 2 — Course Attainment

Combine assessment scores with the mapping matrix to compute final CO attainment and course-wise PO attainment.

CO-PO Mapping & Attainment Workbench

Step 2 Inputs

[Upload file]

[Upload file]

Backend: tfidf v

Model: (default)

Step 1: Mapping

Step 2: Course

Step 3: Semester

Step 4: Program

Step 2 — Course Attainment

CO table is seeded from the Step 1 matrix.

Enter MA / EA / Indirect per CO (editable cells).

Set MA weight, Direct weight, Target level.

Run Attainment Analysis

CO summary + PO summary tables render below

Add course to Step 3

UI mockup of Step 2

The formulas

```
Direct = (MA · w_ma + EA · w_ea) / (w_ma + w_ea)
FinalCO = Direct · w_direct + Indirect · w_indirect
PO = Σ(FinalCO_i · Map_ij) / Σ Map_ij
```

The Direct formula is **normalized** by the sum of internal+external weights, so a 30/50 split produces results on the same scale as the raw assessments. The legacy 40/60 (sums to 1.0) case is unchanged.

Worked example (matches the included spec fixtures)

Quantity	Calculation
Weights	Internal=30%, External=50%, Direct=80%, Indirect=20%
CO1 inputs	MA=2.5, EA=2.8, Indirect=2.7
CO1 Direct	$(2.5 \cdot 0.3 + 2.8 \cdot 0.5) / 0.8 = 2.6875$
CO1 Final	$2.6875 \cdot 0.8 + 2.7 \cdot 0.2 = 2.69$
PO1 course	$(2.69 \cdot 3 + 2.46 \cdot 2 + 2.81 \cdot 1) / 6 = 2.63$

Push forward

After running, the **Add course to Step 3** button appears. Type the course id (e.g. *DBMS*) and credits (e.g. *4*), click it, and Step 3 picks up the course's per-PO row automatically — no re-uploading.

5. Step 3 — Semester PO Attainment

Roll multiple courses up to a single semester score, weighted by course credits.

CO-PO Mapping & Attainment Workbench

Step 3 Inputs

[Upload file]
[Upload file]
Backend: tfidf v
Model: (default)

Step 1: MappingStep 2: CourseStep 3: SemesterStep 4: Program

Step 3 — Semester PO Attainment

Courses pushed from Step 2 appear in the table.
Edit rows or unload courses: semester1.csv.

Run Semester Aggregation

Semester PO table + download button

Add semester to Step 4

UI mockup of Step 3

Formula

$$PO_{sem} = \Sigma(PO_{course} \cdot credits) / \Sigma credits$$

Input shape

```
course_id,credits,P01,P02,P03
DBMS,4,2.63,2.63,2.72
OS,3,2.50,2.40,2.60
```

Empty cells are **skipped** (treated as 'this course doesn't contribute to that PO') — not as zeros that drag the average down.

Worked example

Quantity	Calculation
PO1	$(2.63 \cdot 4 + 2.50 \cdot 3) / 7 = 2.57$
PO2	$(2.63 \cdot 4 + 2.40 \cdot 3) / 7 = 2.53$
PO3	$(2.72 \cdot 4 + 2.60 \cdot 3) / 7 = 2.67$

6. Step 4 — Program PO Attainment

Final stage. Roll every semester up to a single program-level outcome score, credit-weighted across semesters.

CO-PO Mapping & Attainment Workbench

Step 4 Inputs

[Upload file]
[Upload file]
Backend: tfidf v
Model: (default)

Step 1: MappingStep 2: CourseStep 3: SemesterStep 4: Program

Step 4 — Program / Degree PO Attainment

Semesters pushed from Step 3 appear in the table.
Edit rows or [upload semesters_program.csv](#).

Run Program Aggregation

Program PO table + download button

UI mockup of Step 4

Formula

$$PO_program = \Sigma(PO_sem \cdot credits) / \Sigma credits$$

Input shape

```
semester_id,credits,PO1  
Sem1,20,2.50  
Sem2,22,2.58
```

Worked example

Quantity	Calculation
PO1 program	$(2.50 \cdot 20 + 2.58 \cdot 22) / 42 = 2.54$

That's it — the value you'd report for PO1 across the whole program.

7. CLI reference (no UI required)

Every step has a command-line entry point. Outputs are CSVs / JSON written to the --out-dir you pick.

Step 1 — Mapping

```
copo-map \  
  --co-file examples/co.json \  
  --po-file examples/po.json \  
  --out-dir outputs
```

Step 2 — Course attainment

```
copo-attainment \  
  --co-attainment-file examples/co_attainment_spec.json \  
  --mapping-matrix-file examples/mapping_matrix_spec.csv \  
  --config-file examples/attainment_config_spec.json \  
  --out-dir attainment_outputs
```

Step 3 — Semester

```
copo-semester \  
  --courses-file examples/courses_semester1.csv \  
  --out-dir semester_outputs
```

Step 4 — Program

```
copo-program \  
  --semesters-file examples/semesters_program.csv \  
  --out-dir program_outputs
```

Verify the full pipeline at once

Run the four commands in order; the spec example should produce these numbers:

Quantity	Expected value
CO1 Direct	2.6875 (rounded 2.69)
CO1 Final	2.69
Course PO1 / PO2 / PO3	2.63 / 2.63 / 2.72
Semester PO1 / PO2 / PO3	2.57 / 2.53 / 2.67
Program PO1	2.54

8. Troubleshooting

Symptom	What to do
RuntimeError on SBERT/BERT	The model couldn't load. Install nlp deps: <code>pip install -e ".[nlp]"</code> . No silent fallback by design — this is so method comparisons stay fair.
HF Hub rate limits	Set the HF_TOKEN secret in Streamlit Cloud. Unauthenticated downloads still work but throttle on cold start.
Stage 2 numbers look 3× too big	The CO summary's <code>scaled_attainment</code> multiplies by 3 assuming 0–1 inputs. If you pass 0–3 scale inputs (like the spec), read the <code>final_attainment</code> column instead.
Step 3/4 missing a PO	Aggregation skips empty cells. If a course intentionally doesn't touch a PO, leave the cell blank — it won't drag the average down.
Streamlit cold start is slow	First boot installs torch (~700 MB). Allow 3–5 minutes on first deploy; subsequent loads are fast.

Key rule to remember

Every step is a weighted average. Only the weights change — internal/external split, then direct/indirect, then mapping strength, then credits, then credits again.